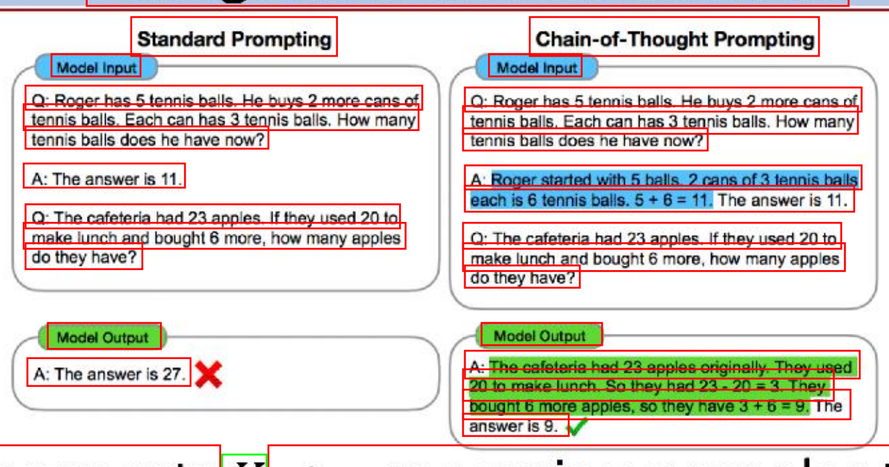
How Do Nonlinear Transformers Acquire Generalization-Guaranteed CoT Ability?

Hongkang Li[1], Meng Wang[1], Songtao Lu[2], Xiaodong Cui[2], Pin-Yu Chen[2]
1:Rensselaer Polytechnic Institute; 2: IBM Research

Overview

- Theoretically characterize how training Transformers can enable chain-of thought (CoT).
- Quantitative analysis of how context examples affect CoT performance.
- Theoretical study of why CoT outperforms ICL.

Background and motivation



- CoT augments K-step reasoning examples to the query for generation, i.e., an extension of in-context learning (ICL) by multi-step examples.
- Existing works [1,2,3]: the expressive power.
- The problem of **why a Transformer can be trained to conduct CoT** is less investigated.

Problem formulation

Study learning K-step reasoning tasks. Each task f is a composition of $\{f_i\}_{i=1}^K$ and outputs $\{z_i\}_{i=1}^K$ for the input $z_0 = x_{query}$, where $z_k = f_k(z_{k-1})$.

Learning model: one-layer Transformer $F(P) = \sum_i W_V \tilde{p}_i softmax\left((W_K \tilde{p}_i)^\top W_Q \tilde{p}_{query}\right)$, where P is the input prompt, $\tilde{p}_i$ is p_i with positional encoding.

Training: Following theoretical works [4,5] for ICL, we aim to solve an empirical risk minimization by stochastic gradient descent with squared loss.

Each Training prompt consists of context examples and a query. Each example $P = \begin{pmatrix} x_1 & y_{i,1} \cdots & y_{i,K-1} \\ y_{i,1} & y_{i,2} \cdots & y_{i,K} \end{pmatrix}$ has K steps, where $y_{i,s} = f_k(y_{i,s-1})$. The query contains the first $k \in [K]$ steps of the reasoning.

CoT Inference: The testing prompt consists of examples and a query. The query contains one step. CoT repeats the two steps below for K steps.

- Greedy decoding to generate the next step.
- Add the generation to the end of the prompt.

CoT generalization error: average 0-1 error of K steps.

ICL Inference: The testing examples only contain the input and the final-step output. The generalization error is the average 0-1 error.

[1] Li et al. Dissecting Chain-of-Thought: Compositionality through In-Context Filtering and Learning. Neurips 2023.
[2] Feng et al. Towards Revealing the Mystery behind Chain of Thought: A Theoretical Perspective. Neurips 2023
[3] Li et al. Chain of Thought Empowers Transformers to Solve Inherently Serial Problem. ICLR 2024.
[4] Huang et al. In-context convergence of transformers. ICML 2024.
[5] Zhang et al. Trained transformers learn linear models in-context. JMLR 2024.

Theoretical results

Data & Tasks: The training and testing data are formulated by training/testing-relevant (TTR/TSR) patterns $\{\mu_i\}_{i=1}^M$ and $\{\mu_i'\}_{i=1}^{M'}$. TSR $\in$ span{TRR}. Consider **erroneous testing prompts**: reasoning in some examples contains incorrect steps. For task f,

- α (or α'): the fraction of contexts with inputs that share the same TRR (or TSR) patterns as the query.
- τ^f: the fraction of accurate context examples.
- ρ^f: the normalized gap of accurate over the most likely inaccurate reasoning steps.

Theorem 1 (Convergence):With the number of context examples and iterations linear in $\Omega(\alpha^{-1})$, the population risk is $\leq O(\epsilon)$.

Theorem 2 (CoT Generalization): To achieve a zero CoT error, the number of context examples for task f is proportional to $(\alpha'\tau^f\rho^f)^{-2}$. The CoT performance is improved with

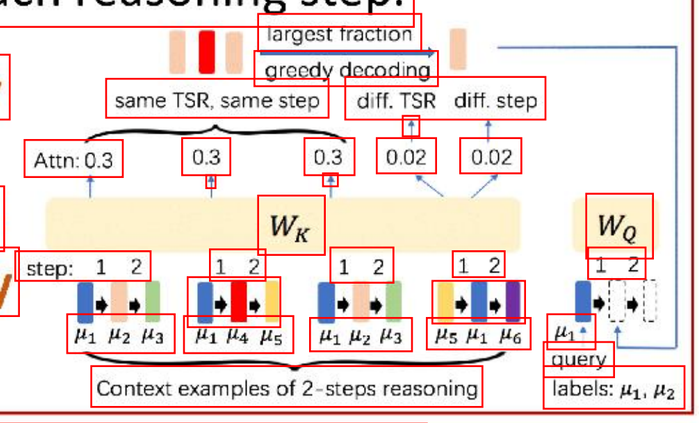
- More accurate reasoning examples,
- More similar examples to the query.

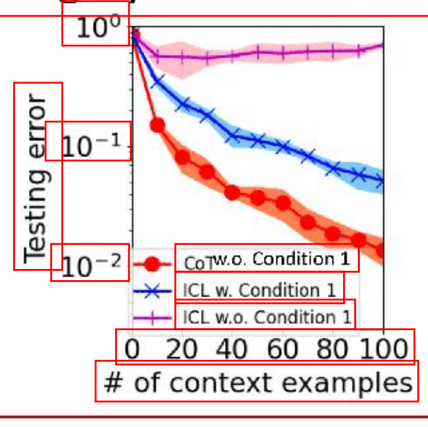
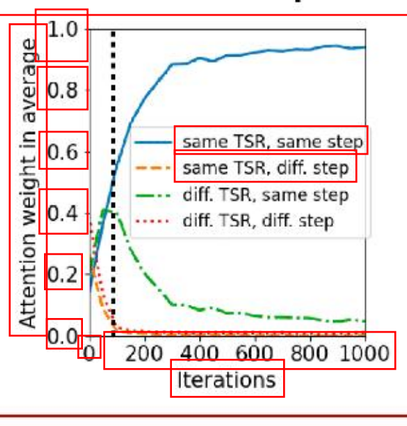
Theorem 3 (Comparison with ICL): The success of ICL requires a dominant fraction of correct input-label examples in the testing prompt (**Condition 1**), but CoT does not.

The mechanism of CoT

Proposition 1: The attention weights is concentrated on prompt columns with the same TSR pattern and the step index as the query step. This leads to a correct prediction in each reasoning step.

Transformers Provably Implement CoT by Attending to the Most Similar Examples Every Step.



Numerical experiments

- Condition 1 is crucial for the success of ICL but not for CoT.
- Training dynamics of CoT: same TSR & step matter.